SIMATS ENGINEERING
Department of Computer Science and Engineering
CSA16 – Data Warehousing and Data Mining
ASSIGNMENT 1 – CO1

Design, Build and Validate a Data Warehouse and OLAP Reporting Solution for a Retail Chain

CO1: Model the data for a data warehouse and apply the various operations that can be performed on the data sets.

Student Name	Register Number	Department / Section	Academic Year / Semester	Faculty Name	Nature of Assignment	Duration	Total Marks	Date of Submission	Date of Evaluation
Harini S	192511153	CSE	2026–2027	Dr.D. Beulah David	Individual	1 Week	50		

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks / Evidence Expected
1	Decision-Support Requirement	4		At least eight strategic/tactical business questions; measures and dimensions; FreshCart-specific OLTP vs DSS justification.
2	Warehouse Architecture	3		Labelled architecture showing sources, staging, ETL, warehouse, metadata, data marts, OLAP and reporting tools; justified architecture choice.
3	Star Schema Design	6		Declared grain; fact measures and additivity; dimensions and hierarchies; keys; conformed Returns fact; schema diagram and DDL.
4	Snowflake Schema Comparison	4		Product and Store snowflake designs; storage calculation; join counts for three queries; performance, ETL and usability comparison; justified choice.
5	ETL Process Design	5		Data-quality issues and transformations; surrogate-key lookup; unknown/late dimensions; SCD choice; load sequence; rejected rows/audit; before-and-after extract.
6	Data Cube and OLAP Operations	5		Cuboid calculations with/without hierarchies; base/apex cuboids; roll-up, drill-down, slice, dice and pivot with FreshCart questions and SQL; materialisation choice.
7	OLAP Application Development	5		Working interactive OLAP application; documented data generation and scale; all five OLAP operations; screenshots, source code and reconciliation.
8	Query Processing Performance	4		Three queries tested on OLTP, star schema and indexed/summary design; execution time, rows read, joins, query plans, storage/load cost and interpretation.
9	KNIME Reporting Workflow	3		Annotated KNIME workflow; node purposes and ETL/OLAP mapping; exported report; limitation at 1,000,000+ rows.
10	Final Design Recommendation	3		Evidence-based recommendation of architecture/schema; comparison with alternatives; trade-offs, load cost, limitations and scalability.
11A	Design Challenge – Impact Analysis	2		Identifies affected and unaffected schema, ETL, cube and architecture components; distinguishes structural, loading and performance impacts.
11B	Design Challenge – Evaluate Original Design	2		Tests original design against 20,000 daily online orders and hourly refresh; identifies adequate areas and limitations.
11C	Design Challenge – Adaptation or Redesign	2		Presents revised schema/ETL flow, product-code mapping, delivery-partner attribute, SCD treatment and incremental/micro-batch loading or aggregates.
11D	Design Challenge – Evidence-Based Defence	2		Supports decision with measured/calculated load time, query time and storage; identifies deployment monitoring metrics.
	TOTAL	**50**		**Marks to be awarded by the faculty.**

OVERALL REMARKS

Faculty feedback:

Student Signature	Faculty Signature	Program Director
Harini. S.		

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present P = partly present ✗ = missing. Mark the status after checking the student's submitted report and supporting evidence.

Task	Evidence Expected	Status	Note
1	Business questions, measures/dimensions and OLTP–DSS analysis	☐	At least eight questions; FreshCart-specific explanation.
2	Warehouse architecture diagram and justification	☐	All required layers, tools and architecture type shown.
3	Star schema, grain, measures, dimensions, keys and DDL	☐	Sales and Returns facts; conformed dimensions demonstrated.
4	Snowflake schema and quantitative comparison	☐	Storage calculations, join counts and design decision included.
5	ETL flow, cleaning rules, SCD decision and cleaned data	☐	All sample defects addressed; rejected rows and audit included.
6	Cube lattice calculations and five OLAP operations	☐	Business questions, cuboids and equivalent SQL included.
7	Working OLAP application and validation evidence	☐	Data sizes, user-selected operations, screenshots, code and reconciliation.
8	Query performance results and query plans	☐	Three queries across three designs; time, rows, joins and storage/load cost.
9	KNIME workflow and exported management report	☐	Annotated nodes and ETL/OLAP concept mapping; limitation discussed.
10	Final design recommendation	☐	Supported by the student's comparisons and measured results.
11A	Impact analysis of changed requirements	☐	Affected and unaffected components identified.
11B	Evaluation under new online-channel workload	☐	20,000 orders/day and hourly refresh considered/tested.
11C	Revised design or adaptation	☐	New attribute, code mapping, SCD and load strategy addressed.
11D	Evidence-based defence and monitoring plan	☐	Measurements/calculations and monitoring metrics included.

IMPLEMENTATION EVIDENCE CHAIN (Tasks 7–9)

Evidence Item	Status	Observation
Warehouse schema and DDL	☐	Star/snowflake structures, keys, constraints and declared grain are documented.
Data source and generation	☐	Real or synthetic data source and generation method documented for 1,000 to 1,000,000+ fact rows.
Application source code	☐	Readable OLAP application code, language, dependencies and execution instructions provided.
Interactive OLAP inputs	☐	User can select operation, dimension, hierarchy level and filter values.
Expected and actual results	☐	Outputs for roll-up, drill-down, slice, dice and pivot are recorded and interpreted.
Reconciliation evidence	☐	Independent totals agree with source data or differences are explained.
Performance test evidence	☐	Queries, data sizes, execution times, rows read, joins and query plans recorded.
Index / summary structure evidence	☐	Additional storage and load cost are recorded and trade-offs explained.
KNIME workflow evidence	☐	Workflow screenshot, node descriptions and exported report included.
Reproducibility and consistency	☐	Code, data, screenshots, tables and report values are consistent and reproducible.

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All Tasks 1–10 and Day-4 Design Challenge 11A–11D answered	☐ Yes ☐ Partly ☐ No
Business questions, architecture and dimensional model are FreshCart-specific	☐ Yes ☐ Partly ☐ No
Fact-table grain, measures, dimensions and keys are clearly stated	☐ Yes ☐ Partly ☐ No
Star and snowflake designs are compared using calculations and examples	☐ Yes ☐ Partly ☐ No
ETL transformations, SCD handling and cleaned sample data are shown	☐ Yes ☐ Partly ☐ No
All five OLAP operations are demonstrated with relevant queries	☐ Yes ☐ Partly ☐ No
OLAP application, source code, screenshots and reconciliation are provided	☐ Yes ☐ Partly ☐ No
Query performance tests include plans, rows read and storage/load costs	☐ Yes ☐ Partly ☐ No
KNIME workflow and exported report are included	☐ Yes ☐ Partly ☐ No
Day-4 challenge addresses online orders, hourly refresh and changing customer details	☐ Yes ☐ Partly ☐ No
All diagrams, tables, calculations and references are numbered and readable	☐ Yes ☐ Partly ☐ No
No unfilled template placeholders remain in the student report	☐ Yes ☐ Partly ☐ No
Viva recommended where implementation, calculations or results need clarification	☐ Yes ☐ No

Note: This DWDM assignment evaluation template is based on Assignment 1 – CO1. Blank marks and evidence statuses are intentionally left for faculty evaluation.

ASSUMPTIONS AND DESIGN BASIS

The answers below use the FreshCart scenario and requirements stated in the assignment. Where the question does not provide actual sales data, row widths, hardware or benchmark results, clearly stated design assumptions and illustrative calculations are used. Performance values are not presented as measured results; they must be replaced with timings from the student's own execution.

- FreshCart operates 120 supermarkets in six states and has a normalised OLTP database.
- The sales fact grain is one row per product line in an order, not one row per complete order.
- Currency is standardised to INR for reporting. USD amounts are converted using the exchange rate effective on the transaction date; the example below assumes ₹83 per USD only to demonstrate the rule.
- A warehouse surrogate key is used for every dimension. Unknown dimension members are represented by a special key, normally 0.
- The preferred design is a central enterprise data warehouse with dependent data marts and a star schema for reporting.

TASK 1 – ANALYSE THE DECISION-SUPPORT REQUIREMENT (4 MARKS)

1.1 Why the existing OLTP system is unsuitable

FreshCart's operational database is designed to record day-to-day transactions such as billing, order entry, returns and loyalty-card swipes. It is highly normalised into tables such as Orders, OrderLines, Customers, Products, Stores, Employees, Promotions, Suppliers and Returns. This structure reduces duplication and supports fast inserts and updates. However, it is not suitable for repeatedly scanning and combining years of data for management decisions.

- Operational interference: large analytical queries compete with billing transactions for CPU, memory, disk I/O and database locks. This explains why billing counters slow down at peak hours.
- Complex joins: a sales analysis may need Orders, OrderLines, Products, Stores, Customers, Promotions and Returns. Joining many normalised tables for every report increases query complexity and execution cost.
- Inconsistent business definitions: departments calculate net sales differently. A warehouse can define net sales once, for example gross line value minus discounts and refunds, and reuse the certified measure.
- Historical analysis: operational customer and product records are frequently updated. Without history management, the OLTP database may not show the address, loyalty tier or product category that applied when an earlier sale occurred.
- Different query pattern: OLTP handles many short, selective transactions; management analysis scans and aggregates large numbers of rows across months, stores, regions and categories.

Therefore, FreshCart needs a separate decision-support environment that receives integrated, cleaned and historical data from operational systems and supports read-intensive analysis without slowing down billing.

1.2 Business questions, measures and dimensions

No.	Business question	Type	Measures	Dimensions
1	Which product categories show declining net sales quarter over quarter in each region?	Strategic	Net sales; sales growth %	Time (quarter), Product (category), Store (region)
2	Which promotions increased average basket value, and in which stores?	Tactical	Average basket value; order count; net sales	Promotion, Store, Time

3	How does weekend sales performance differ from weekday performance for each department?	Tactical	Net sales; quantity sold; order count	Date (day type), Product (department), Store
4	Which customer segments return the most items, and of which products?	Strategic	Returned quantity; return value; return rate	Customer segment, Product, Store, Time
5	What is each store's year-to-date sales compared with the same period last year?	Strategic	YTD net sales; previous-year YTD sales; variance	Time, Store, Region
6	Which stores have the highest sales per square metre?	Strategic	Net sales; store area; sales per sq. metre	Store, Time, Product department
7	Which products are frequently purchased together?	Tactical	Basket count; product-pair count; support	Product, Store, Time, Customer segment
8	Which suppliers are associated with products having high return rates?	Strategic	Returned quantity; sold quantity; return rate	Supplier, Product, Store, Time
9	How many orders and how much revenue come from each sales channel?	Tactical	Order count; gross sales; net sales	Channel, Time, Store/fulfilment location
10	Which stores are below their monthly sales target?	Tactical	Actual net sales; target; achievement %	Store, Time, Region

Strategic questions support long-term decisions such as regional expansion, category planning and supplier strategy. Tactical questions support near-term actions such as promotion adjustment, store staffing and monthly target correction.

1.3 OLTP and decision-support system comparison

Aspect	FreshCart OLTP system	FreshCart decision-support warehouse
Purpose	Record sales, returns, customer changes and billing accurately.	Analyse trends, compare performance and support management decisions.
Data currency	Current operational state; updated continuously.	Integrated historical data; refreshed on a schedule, initially daily or hourly as required.
Structure	Normalised transaction tables to reduce update anomalies.	Dimensional star/snowflake structures with facts and dimensions.
Query pattern	Short INSERT, UPDATE, DELETE and selective SELECT statements.	Long SELECT statements, joins, grouping, filtering and aggregations.
Users	Cashiers, store staff, operational applications and customer-service staff.	Managers, analysts, executives and BI/reporting users.
Volume	Large number of small concurrent transactions.	Large scans across months/years and millions of sales lines.
FreshCart impact	Analytical scans slow billing and reports use inconsistent definitions.	Separates analytical load and applies common definitions for net sales and returns.

1.4 Questions that cannot be answered efficiently in OLTP

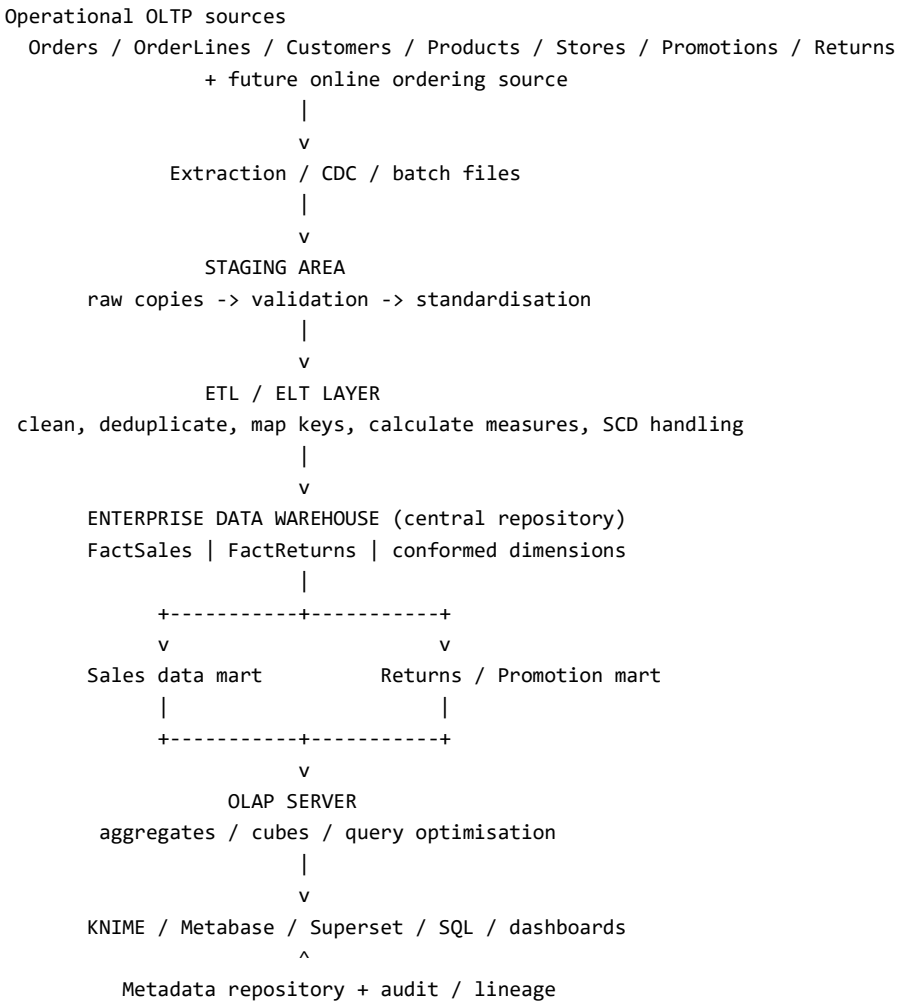
Quarter-over-quarter category trends, year-to-date comparisons with the previous year, return rates by historical customer segment, and promotion impact across stores are inefficient in the OLTP system. They require historical snapshots, multiple joins and aggregation over many transaction rows. The operational system may overwrite a customer's address or loyalty tier and therefore lose the historical context. Running such queries

during peak billing also affects transaction response time. The warehouse solves these issues through historical dimensions, conformed definitions and pre-aggregated reporting structures.

Conclusion: A separate decision-support warehouse is justified because it protects transaction performance, integrates multiple sources, preserves history and provides consistent measures for all departments.

TASK 2 – PROPOSE THE WAREHOUSE ARCHITECTURE (3 MARKS)

2.1 Recommended architecture



Metadata repository stores table definitions, source-to-target mappings, data types, business definitions (such as net sales), refresh time, data lineage, quality rules and cube hierarchies. It is used by ETL developers and analysts to understand and maintain the warehouse.

2.2 Architecture choice and justification

Choose an enterprise data warehouse (EDW) with dependent data marts. The EDW integrates all 120 stores and the future online channel into one governed repository. Dependent marts are created from the EDW for sales, returns and promotions. This prevents each department from building an independent definition of sales or customer segment.

- Independent data marts are not preferred because each department could store different customer, product and net-sales definitions, recreating the current inconsistency.

- A virtual warehouse is not preferred as the only solution because repeated queries would still access operational sources and could affect billing performance. It also provides weaker historical storage unless additional systems are introduced.
- A single EDW with dependent marts costs more to establish initially, but it provides consistent dimensions, controlled security, lineage and scalable reporting.

The architecture supports batch loading initially and can add change-data-capture (CDC) or micro-batch ingestion for hourly refreshes.

TASK 3 – DESIGN THE STAR SCHEMA (6 MARKS)

3.1 Business process and grain

Business process: retail sales measurement. Grain: one row in FactSales represents one product line within one customer order at one store on one date under one promotion (or the default no-promotion member). Quantity and amount measures are recorded at this line-item level. The grain must be declared before defining measures because it determines what one fact row means and prevents double counting.

3.2 Fact table and measures

Measure	Definition	Additivity
QuantitySold	Number of units sold on the order line.	Additive across product, store, customer and time.
GrossSalesAmount	Quantity × unit selling price before discount.	Additive.
DiscountAmount	Discount allocated to the order line.	Additive.
NetSalesAmount	Gross sales − discount − allocated refund/adjustment, according to the approved policy.	Additive when stored at line grain.
UnitSellingPrice	Selling price per unit.	Non-additive; use weighted average, not SUM.
ProfitAmount	Net sales − cost of goods sold.	Additive.
MarginPercentage	Profit ÷ net sales × 100.	Non-additive; recompute from summed numerator and denominator.
OrderCount	Normally derived as COUNT(DISTINCT OrderID), not summed from duplicated line rows.	Non-additive across product lines.

Net sales definition: NetSalesAmount = GrossSalesAmount − DiscountAmount − RefundAmount. Returns should be stored in a separate return fact table and linked to the original sale where possible. If refunds are included in net sales, the business rule must prevent subtracting them twice.

3.3 Dimension tables and hierarchies

Dimension	Example attributes	Concept hierarchy
DimDate	DateKey, FullDate, Day, DayName, Week, Month, MonthName, Quarter, Year, IsWeekend	Day → Month → Quarter → Year
DimProduct	ProductKey, ProductCode, ProductName, Brand, Category, Department, UnitSize	Product → Category → Department
DimStore	StoreKey, StoreCode, StoreName, City, State, Region, StoreType	Store → City → State → Region
DimCustomer	CustomerKey, CustomerID, CustomerName, Segment, Gender (if permitted), LoyaltyTier, Address, Effective dates	Customer → Segment; address and tier tracked historically
DimPromotion	PromotionKey, PromotionCode,	Promotion → PromotionType

	PromotionName, PromotionType, StartDate, EndDate, DiscountType	
DimEmployee (optional)	EmployeeKey, EmployeeID, EmployeeName, Role	Employee → Role

3.4 Keys and star schema diagram

Each dimension has a surrogate primary key generated by the warehouse (for example ProductKey). The original business identifier (ProductCode or CustomerID) is retained as a natural key for source matching. FactSales contains foreign keys to all relevant dimensions. DateKey, ProductKey, StoreKey, CustomerKey and PromotionKey together identify the dimensional context of a sale line; OrderID and OrderLineNumber are retained as degenerate transaction identifiers.

```
                DimDate
                   |
DimProduct ---- FactSales ---- DimStore
```

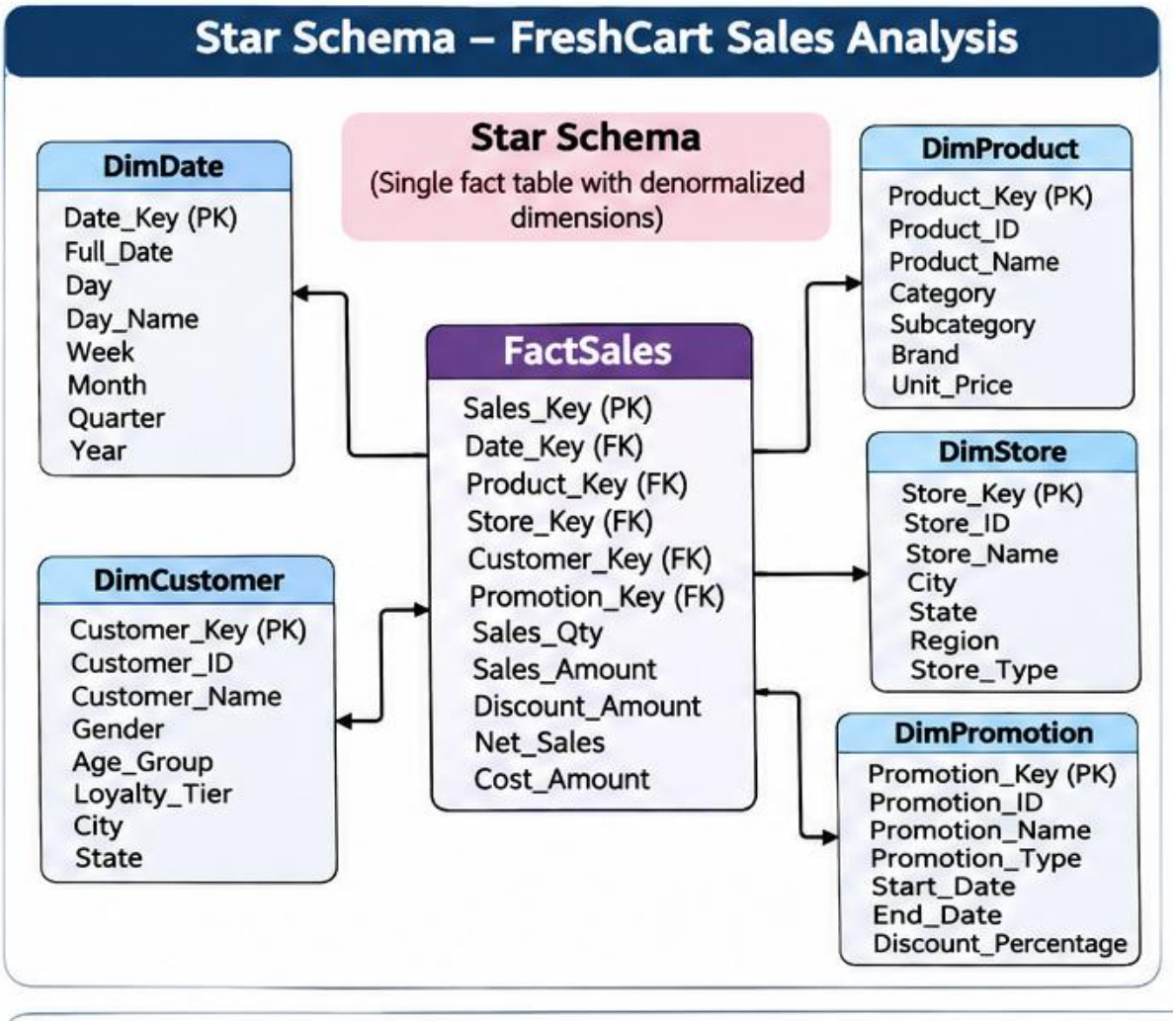


Figure 1: Star schema for FreshCart sales analysis.

```
      |
DimCustomer
      |
DimPromotion
      |
```

```
              DimEmployee (optional)

   FactReturns shares Date, Product, Store and Customer dimensions.
```

3.5 Example SQL DDL

```
CREATE TABLE DimDate (
  DateKey INT PRIMARY KEY, FullDate DATE NOT NULL,
  DayNo INT, DayName VARCHAR(12), MonthNo INT, MonthName VARCHAR(12),
  QuarterNo INT, YearNo INT, IsWeekend BOOLEAN
);
CREATE TABLE DimProduct (
  ProductKey INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  ProductCode VARCHAR(40), ProductName VARCHAR(120),
  Brand VARCHAR(60), Category VARCHAR(60), Department VARCHAR(60),
  UnitSize VARCHAR(30), IsCurrent BOOLEAN DEFAULT TRUE
);
CREATE TABLE DimStore (
  StoreKey INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  StoreCode VARCHAR(30), StoreName VARCHAR(100),
  City VARCHAR(60), State VARCHAR(60), Region VARCHAR(40)
);
CREATE TABLE DimCustomer (
  CustomerKey INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  CustomerID VARCHAR(40), CustomerName VARCHAR(100),
  Segment VARCHAR(40), LoyaltyTier VARCHAR(30),
  Address VARCHAR(250), EffectiveFrom DATE, EffectiveTo DATE,
  IsCurrent BOOLEAN
);
CREATE TABLE DimPromotion (
  PromotionKey INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  PromotionCode VARCHAR(40), PromotionName VARCHAR(100),
  PromotionType VARCHAR(40), StartDate DATE, EndDate DATE
);
CREATE TABLE FactSales (
  SalesLineKey BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  OrderID VARCHAR(40), OrderLineNumber INT,
  DateKey INT REFERENCES DimDate(DateKey),
  ProductKey INT REFERENCES DimProduct(ProductKey),
  StoreKey INT REFERENCES DimStore(StoreKey),
  CustomerKey INT REFERENCES DimCustomer(CustomerKey),
  PromotionKey INT REFERENCES DimPromotion(PromotionKey),
  QuantitySold DECIMAL(12,2), GrossSalesAmount DECIMAL(14,2),
  DiscountAmount DECIMAL(14,2), NetSalesAmount DECIMAL(14,2),
  CostAmount DECIMAL(14,2), ProfitAmount DECIMAL(14,2)
);
CREATE TABLE FactReturns (
  ReturnLineKey BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  ReturnID VARCHAR(40), OriginalOrderID VARCHAR(40),
  DateKey INT REFERENCES DimDate(DateKey),
  ProductKey INT REFERENCES DimProduct(ProductKey),
  StoreKey INT REFERENCES DimStore(StoreKey),
  CustomerKey INT REFERENCES DimCustomer(CustomerKey),
  ReturnQuantity DECIMAL(12,2), ReturnAmount DECIMAL(14,2)
);
```

The SQL uses identity syntax supported by several relational databases; syntax may need minor adjustment for MySQL, PostgreSQL or SQL Server. In production, add NOT NULL constraints, indexes, unique source-key constraints, check constraints and a special unknown row in each dimension.

3.6 Shared dimensions

FactSales and FactReturns share conformed dimensions such as Date, Product, Store and Customer. This allows management to compare sold quantity and returned quantity using the same product, store, date and customer definitions. FactReturns has a different grain: one row per returned product line. Measures from the two fact tables should be aggregated separately before joining to avoid multiplying rows.

TASK 4 – COMPARE WITH A SNOWFLAKE SCHEMA (4 MARKS)

4.1 Snowflake design

In the snowflake design, descriptive hierarchy attributes are separated into normalised tables. Product is connected to Brand, Category and Department. Store is connected to City, State and Region.

```
DimProduct(ProductKey, ProductCode, ProductName, BrandKey, CategoryKey, DepartmentKey)
  -> DimBrand(BrandKey, BrandName)
  -> DimCategory(CategoryKey, CategoryName)
  -> DimDepartment(DepartmentKey, DepartmentName)
DimStore(StoreKey, StoreCode, StoreName, CityKey)
```

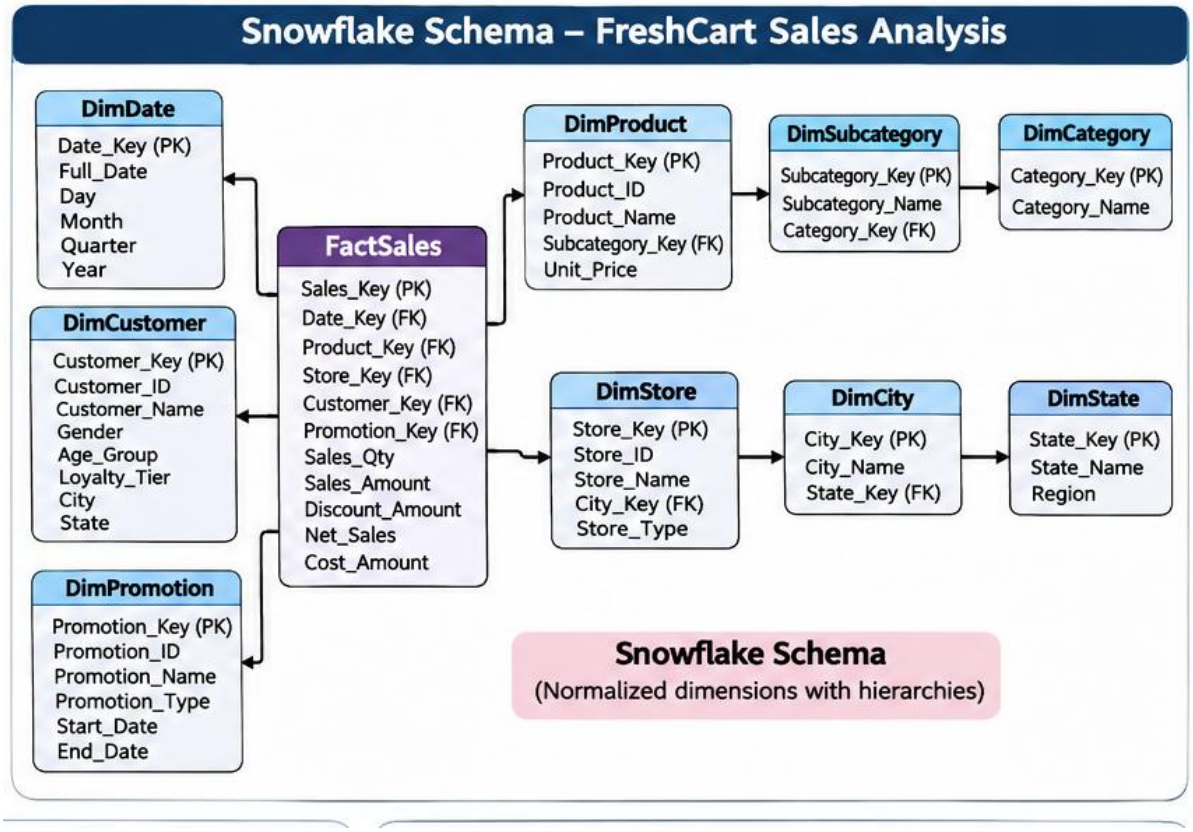


Figure 2: Snowflake schema with normalised product and store hierarchies.

```
-> DimCity(CityKey, CityName, StateKey)
-> DimState(StateKey, StateName, RegionKey)
-> DimRegion(RegionKey, RegionName)
```

4.2 Estimated storage calculation

The following is a transparent illustrative estimate. It assumes average stored attribute widths and excludes database page overhead, compression and indexes. Actual storage must be measured on the selected DBMS.

Dimension design	Calculation	Approx. storage
Star Product	50,000 × 136 bytes (product plus repeated brand/category/department text)	6,800,000 bytes = 6.80 MB
Snowflake Product base	50,000 × 88 bytes	4,400,000 bytes = 4.40 MB
Snowflake Brand	800 × 24 bytes	19,200 bytes
Snowflake Category	60 × 24 bytes	1,440 bytes
Snowflake Department	12 × 24 bytes	288 bytes
Snowflake Product total	4,400,000 + 19,200 + 1,440 + 288	4.421 MB approx.
Star Store	120 × 129 bytes	15,480 bytes
Snowflake Store + City + State + Region	120×58 + 70×24 + 6×24 + 3×24	8,856 bytes

Estimated combined saving = (6.80 MB + 0.01548 MB) − (4.421 MB + 0.008856 MB) ≈ 2.386 MB. Percentage saving ≈ 2.386 / 6.81548 × 100 = 35.0%. This is a modest absolute saving because there are only 120 stores and the assumed product text widths are limited. With wider repeated descriptions or larger dimensions, savings can be greater.

4.3 Join-count comparison

Representative query	Star schema joins	Snowflake joins
Net sales by product category and region	FactSales→Product, Store, Date: 3 joins; category and region are attributes in dimensions.	FactSales→Product→Category and Store→City→State→Region plus Date: up to 7 joins.
Sales by department and state	3 joins (FactSales to Product, Store and Date).	Product→Department and Store→City→State: 5 joins including Date.
Sales by brand and city	3 joins (FactSales to Product, Store and Date).	Product→Brand and Store→City: 5 joins including Date.

4.4 Performance, ETL and usability

- Star schema: fewer joins, simpler SQL, easier for business users and usually faster for interactive dashboards. It repeats descriptive text, which can increase storage and update work.
- Snowflake schema: reduces repeated hierarchy attributes and centralises maintenance. However, it introduces joins, more complex ETL dependencies and a less intuitive model for non-technical users.

Recommendation: retain the star schema for FreshCart's primary reporting dimensions because fast, understandable analysis is more valuable than the relatively small estimated storage saving. Use snowflaking only if dimension growth, repeated large attributes or governance requirements make storage and centralised hierarchy maintenance significant.

TASK 5 – DESIGN THE ETL PROCESS (5 MARKS)

5.1 ETL flow

```
Source OLTP / files -> Extract -> Raw staging -> Profile and validate
 -> Standardise names, dates, currency and product codes
 -> Deduplicate / resolve customer and product identities
 -> Lookup or create dimension surrogate keys
 -> Load dimensions (Date, Product, Store, Customer, Promotion)
 -> Load FactSales / FactReturns -> Reconcile -> Publish
```

The raw extract is retained unchanged for audit. A staging layer stores source file name, extraction timestamp, source system and batch ID. Transformation rules are applied before fact loading.

5.2 Data-quality problems and transformation rules

Input issue	Problem	Rule / treatment
Rajesh Kumar vs R. Kumar	Possible duplicate customer identity; name alone is not a safe key.	Match using source CustomerID/loyalty ID. If absent, send to identity-resolution review; do not automatically merge on initials.
Chennai-04 vs chennai-04	Inconsistent case.	Trim spaces and standardise case; map through a StoreCode cross-reference to one StoreKey.
Basmati Rice 5kg vs Basmati rice 5 kg	Product naming/spacing inconsistency.	Map source product code first; otherwise normalise case/spacing and use an approved product cross-reference. Do not rely only on fuzzy text.
Qty = -3	Negative quantity may represent a return, not a sale.	Reject from FactSales and route to FactReturns if return reference and business rule validate it. Preserve source value in reject/audit table.
03/11/2026 and 2026-11-03	Ambiguous/inconsistent date formats.	Parse using source-specific locale; standardise to ISO date 2026-11-03. Quarantine ambiguous dates.
04-Nov-26 and 04/11/2026	Different date formats.	Parse with explicit format rules; both resolve to 2026-11-04 if DD/MM is the agreed locale.
(null) Store	Missing mandatory store reference.	Use StoreKey=0 (Unknown Store) temporarily and create a late-arriving dimension exception for correction.
9.40 (USD)	Currency differs from warehouse reporting currency.	Extract numeric amount 9.40; convert using transaction-date FX rate. At assumed ₹83/USD, INR unit price = ₹780.20. Store original currency and FX rate for audit.
(null) Product	Missing mandatory product.	Use ProductKey=0 temporarily and queue for product master resolution; do not invent a product.

5.3 Cleaned sample extract

The following output shows the expected standardised fields. Customer identity for Rajesh Kumar and R. Kumar remains unmerged unless a stable customer identifier confirms they are the same person. The negative quantity is routed to the returns flow. The row with missing product is held for correction or loaded with the unknown key according to the agreed policy.

Order	Customer (resolved display)	Store	Product	Qty	Unit price (INR)	Order date	Disposition
1001	Rajesh Kumar	CHENNAI-04	BASMATI RICE 5KG	2	620.00	2026-11-03	Sales candidate
1002	R. Kumar	CHENNAI-04	BASMATI RICE 5KG	1	620.00	2026-11-03	Sales candidate; identity check
1003	Anita Joseph	PUNE-11	OLIVE OIL 1L	-3	780.00	2026-11-04	Route to returns; validate original sale
1004	Meena S	UNKNOWN STORE	TEA 250G	4	95.50	2026-11-04	Unknown store key; resolve later
1005	Anita Joseph	PUNE-11	OLIVE OIL 1L	3	780.20	2026-11-04	Convert USD at illustrative ₹83/USD

1006	Vikram Rao	HYD-02	UNKNOWN PRODUCT	1	1250.00	2026-11-05	Unknown product key; resolve later

5.4 Surrogate-key lookup and late-arriving dimensions

For each cleaned business key, ETL looks up the corresponding dimension row and retrieves its surrogate key. If a valid member is not yet available, the fact may reference the special Unknown key (0), while an exception record stores the source key, batch ID and reason. When the real dimension arrives, ETL updates the fact foreign key from 0 to the correct surrogate key. For a late-arriving fact whose dimension details are available in the source event, a provisional dimension member can be created and completed later.

5.5 Slowly Changing Dimension choice

Use SCD Type 2 for Customer address and loyalty tier because management needs to report sales by the address and tier that applied at the time of the order. When either attribute changes, expire the old row by setting EffectiveTo and IsCurrent=false, then insert a new row with a new CustomerKey, EffectiveFrom and IsCurrent=true. Fact rows retain the CustomerKey valid at the sale date. Type 1 would overwrite history and cannot answer historical questions; Type 3 stores only limited previous values.

5.6 Initial load, incremental load and audit

- Initial load: extract the full master and transaction history, profile and clean it, load dimensions first, then load facts after surrogate-key resolution. Build indexes and aggregates after bulk loading where appropriate.
- Incremental load: use CDC, a reliable last-updated timestamp or source sequence number to capture only new/changed records. Run hourly micro-batches for the online channel; make the process idempotent using source transaction keys and batch IDs.
- Load order: populate reference/date dimensions; load Product, Store, Customer and Promotion dimensions; apply SCD changes; resolve keys; then load FactSales and FactReturns; finally refresh aggregates and dashboards.
- Rejected rows: store original row, source, batch ID, rejection reason and timestamp in an ETL_Reject table. Do not silently discard invalid data.
- Audit: compare extracted, accepted, rejected and loaded counts. Reconcile sales quantity and monetary totals by source and batch. Record start/end time, row counts, error counts and status in ETL_Audit.

Conclusion: The ETL process standardises inconsistent source data, preserves customer history, resolves keys and provides an auditable route for incomplete records.

TASK 6 – MODEL THE DATA CUBE AND APPLY OLAP OPERATIONS (5 MARKS)

6.1 Cuboid lattice calculation

The four dimensions are Time (day→month→quarter→year), Product (product→category→department), Store (store→city→state) and Customer Segment (one level). Including the ALL level for each dimension, the number of hierarchy-aware cuboids is:

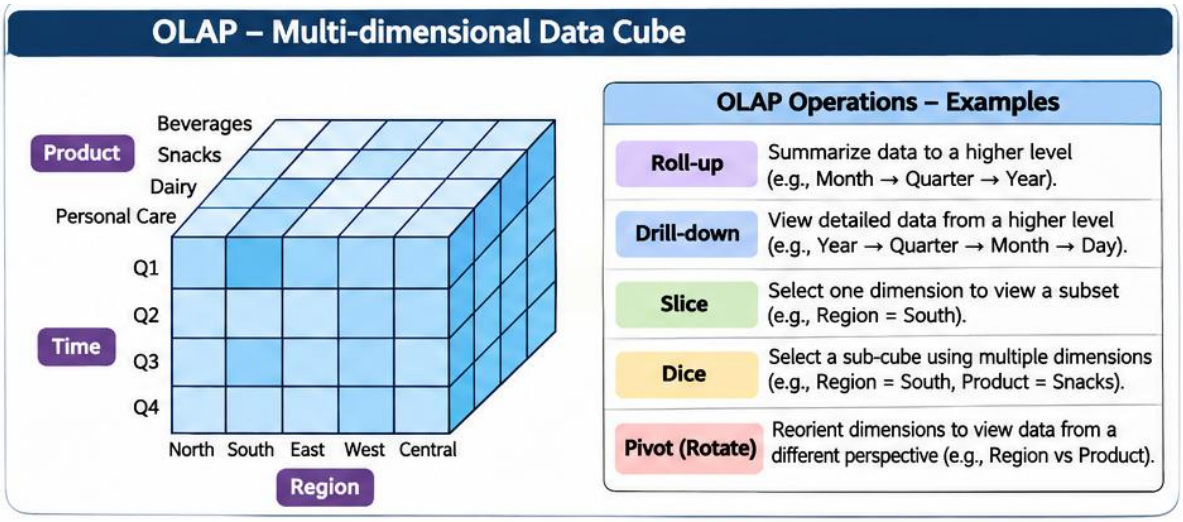


Figure 3: OLAP data cube and roll-up, drill-down, slice, dice and pivot operations.

Time choices = 4 levels + ALL = 5; Product choices = 3 + ALL = 4; Store choices = 3 + ALL = 4; Segment choices = 1 + ALL = 2.

Total cuboids = $5 \times 4 \times 4 \times 2 = 160$.

If hierarchies are ignored and each of the four dimensions is treated simply as present or aggregated to ALL, the lattice has $2^4 = 16$ cuboids. The hierarchy levels increase the number of possible aggregation combinations.

6.2 Base and apex cuboids

- Base cuboid: Day × Product × Store × Customer Segment. It contains the most detailed sales aggregates at the lowest available level of every dimension.
- Apex cuboid: ALL × ALL × ALL × ALL. It contains the grand total, such as total net sales for the complete dataset.

6.3 OLAP operations with FreshCart questions and SQL

Assume a star schema with FactSales joined to DimDate, DimProduct, DimStore and DimCustomer. SQL examples use generic SQL syntax.

A. Roll-up

Question: What are total net sales by region and year, rather than by individual store and day? Roll-up moves from Store to Region and Day to Year, producing a more summarised cuboid.

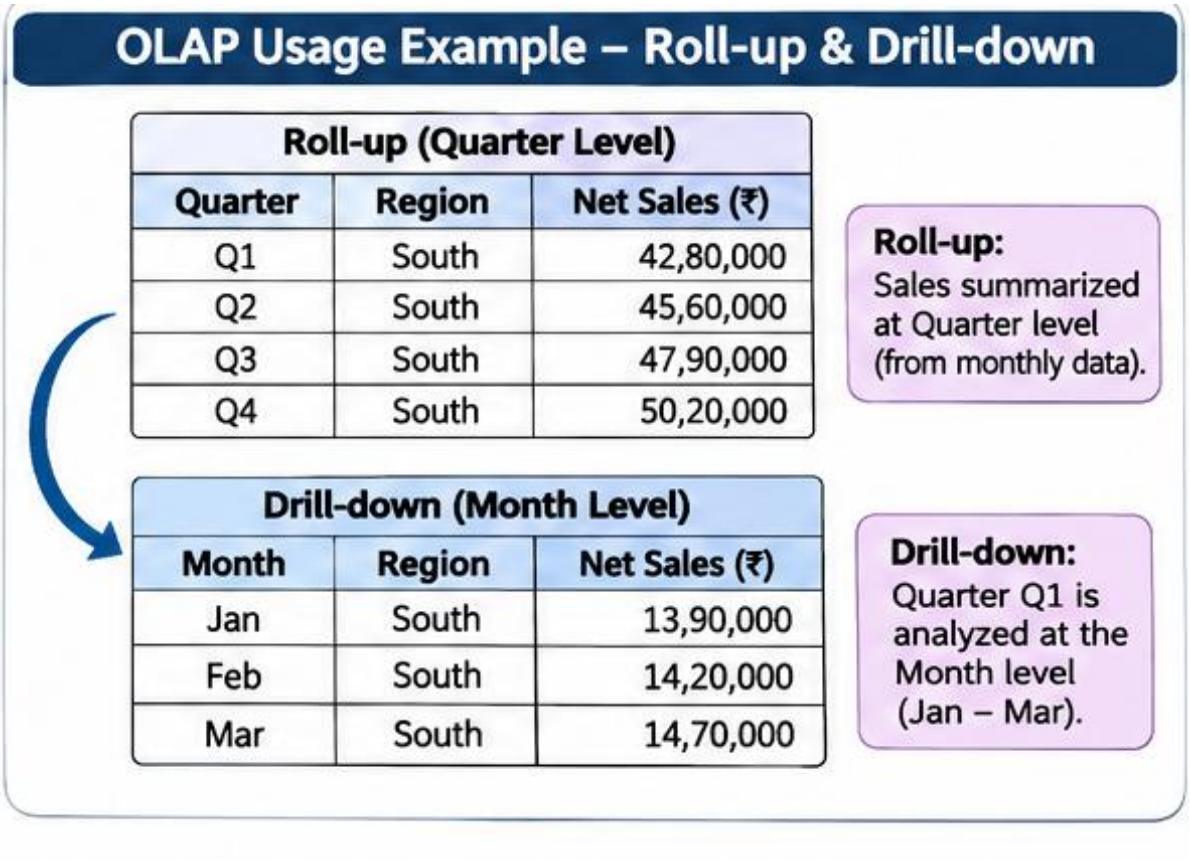


Figure 4: FreshCart example showing quarterly roll-up and monthly drill-down.

```
SELECT s.Region, d.YearNo, SUM(f.NetSalesAmount) AS NetSales
FROM FactSales f JOIN DimStore s ON f.StoreKey=s.StoreKey
JOIN DimDate d ON f.DateKey=d.DateKey
GROUP BY s.Region, d.YearNo;
```

B. Drill-down

Question: A manager sees low annual sales in South region. Which state and city caused the decline? Drill-down moves from Region to State/City and Year to Quarter or Month.

```
SELECT s.State, s.City, d.QuarterNo, SUM(f.NetSalesAmount) AS NetSales
FROM FactSales f JOIN DimStore s ON f.StoreKey=s.StoreKey
JOIN DimDate d ON f.DateKey=d.DateKey
WHERE s.Region='South' AND d.YearNo=2026
GROUP BY s.State, s.City, d.QuarterNo;
```

C. Slice

Question: What is the sales performance for the year 2026 across all products, stores and segments? Slice fixes one dimension to one value.

```
SELECT p.Department, s.Region, SUM(f.NetSalesAmount) AS NetSales
FROM FactSales f JOIN DimDate d ON f.DateKey=d.DateKey
JOIN DimProduct p ON f.ProductKey=p.ProductKey
JOIN DimStore s ON f.StoreKey=s.StoreKey
WHERE d.YearNo=2026
GROUP BY p.Department, s.Region;
```

D. Dice

Question: Compare selected grocery departments in Chennai and Pune stores for Q1 and Q2 of 2026, for Gold and Silver customer segments. Dice filters multiple dimensions to a sub-cube.

```
SELECT p.Department, s.City, c.Segment, d.QuarterNo,
       SUM(f.NetSalesAmount) AS NetSales
FROM FactSales f JOIN DimProduct p ON f.ProductKey=p.ProductKey
JOIN DimStore s ON f.StoreKey=s.StoreKey
JOIN DimCustomer c ON f.CustomerKey=c.CustomerKey
JOIN DimDate d ON f.DateKey=d.DateKey
WHERE p.Department IN ('Grocery','Beverages')
  AND s.City IN ('Chennai','Pune') AND d.YearNo=2026
  AND d.QuarterNo IN (1,2) AND c.Segment IN ('Gold','Silver')
GROUP BY p.Department, s.City, c.Segment, d.QuarterNo;
```

E. Pivot

Question: For each department, show quarterly sales as columns so that managers can compare Q1–Q4. Pivot changes the orientation of the report; it does not change the underlying measures.

```
SELECT p.Department,
 SUM(CASE WHEN d.QuarterNo=1 THEN f.NetSalesAmount ELSE 0 END) AS Q1,
 SUM(CASE WHEN d.QuarterNo=2 THEN f.NetSalesAmount ELSE 0 END) AS Q2,
 SUM(CASE WHEN d.QuarterNo=3 THEN f.NetSalesAmount ELSE 0 END) AS Q3,
 SUM(CASE WHEN d.QuarterNo=4 THEN f.NetSalesAmount ELSE 0 END) AS Q4
FROM FactSales f JOIN DimProduct p ON f.ProductKey=p.ProductKey
JOIN DimDate d ON f.DateKey=d.DateKey
WHERE d.YearNo=2026 GROUP BY p.Department;
```

6.4 Partial materialisation strategy

If only a small fraction of the 160 cuboids can be stored, materialise high-use summaries rather than every possible combination. Recommended cuboids are:

- Year × Region × Department × Segment: supports executive trend and strategic planning.
- Month × Store × Department: supports store and department performance dashboards.
- Quarter × Region × Category: supports quarter-over-quarter category decline analysis.
- Month × Promotion × Store: supports promotion effectiveness and average basket analysis.

These cuboids are useful because they answer frequent management questions and reduce repeated aggregation over the detailed fact table. The base cuboid remains stored as the detailed source of truth. Refresh cost, storage and query frequency should be measured before adding more summaries.

TASK 7 – DEVELOP AN OLAP APPLICATION (5 MARKS)

7.1 Proposed application design

Build a small Python application using Flask and SQL (or use Metabase/Apache Superset). The user interface should allow the analyst to choose an OLAP operation, dimension, hierarchy level, date range, region, store, department and customer segment. The application must generate a query based on these selections rather than relying only on fixed queries.

Component	Purpose
Flask web interface	Accepts operation, dimension level and filter selections.
Python query service	Validates selections and constructs parameterised SQL.
Warehouse database	Stores FactSales, FactReturns and dimensions.
Chart/report layer	Displays grouped results as tables, bar charts or line charts.
Reconciliation module	Compares warehouse totals with independently aggregated

	source totals.

7.2 Operations supported

Operation	User interaction	Expected output
Roll-up	Choose Day→Month or Store→Region.	Summarised sales at selected higher level.
Drill-down	Select a result and request lower level.	More detailed sales by month, city or store.
Slice	Choose one fixed value, e.g. Year=2026.	Report restricted to the selected year.
Dice	Choose multiple values across dimensions.	Filtered sub-cube for selected stores, categories and segments.
Pivot	Choose row dimension and column dimension.	Cross-tab report with dimensions exchanged.

7.3 Data generation and scale testing

Generate reproducible synthetic data using a fixed random seed. Create dimension records first, then create order headers and line items with valid foreign keys, realistic dates, products, stores, customers and promotions. Generate separate datasets or batches at 1,000, 10,000, 100,000 and 1,000,000 fact rows. Record the seed, script version, row count and generation time. Avoid random invalid foreign keys unless deliberately testing data-quality handling.

```
import random
random.seed(42)  # reproducible test data
# Select valid DateKey, ProductKey, StoreKey, CustomerKey and PromotionKey
# Generate QuantitySold, UnitPrice, Discount and NetSalesAmount
# Insert rows in batches and record elapsed time
```

7.4 Validation and reconciliation

For each dataset size, independently calculate source net sales using the agreed formula and compare it with the warehouse grand total. The reconciliation difference should be zero, apart from documented currency rounding or rejected records. Also compare row counts, quantities and totals by store and month. Capture screenshots of the selected operation, filters, result table and chart as evidence.

7.5 Sample application logic

```
operation = request.args.get('operation')
level = request.args.get('level')
region = request.args.get('region')
# Whitelist dimensions and levels; never concatenate raw user SQL.
# Build parameterised SELECT/GROUP BY clauses from approved mappings.
# Execute against the warehouse and return rows as JSON.
```

Security note: use parameterised queries for filter values and a whitelist for column names and hierarchy levels. Do not allow users to submit arbitrary SQL. The application should also show the refresh timestamp and selected filters.

Evidence to attach in the final submission: screenshots of all five operations, the source code, data-generation script, database row counts and reconciliation output. The written design above is an implementation plan; a working application and screenshots must be produced by running the code.

TASK 8 – EVALUATE QUERY PROCESSING PERFORMANCE (4 MARKS)

8.1 Three representative queries

- Q1: Net sales by region and quarter – joins sales lines to store and date.
- Q2: Net sales by product department and customer segment – joins product and customer dimensions.
- Q3: Return rate by product category and region – aggregates FactReturns and FactSales at matching dimensional levels.

8.2 Test configurations

Configuration	Description	Expected characteristics
(i) OLTP-style schema	Orders, OrderLines, Products, Stores, Customers and Promotions joined at query time.	Many joins; normalised design; transaction indexes may not suit broad scans.
(ii) Star schema	FactSales joined to denormalised dimensions.	Fewer joins; simpler group-by; designed for analytical scans.
(iii) Star + indexes/summary	Star schema with foreign-key/date indexes and selected aggregate tables.	Faster common reports; extra storage and ETL refresh cost.

8.3 Benchmark method and result recording

Run each query on the same machine, database version, cache state and dataset sizes. Use 1,000, 10,000, 100,000 and 1,000,000 fact rows. Run each query several times and report the median. Record execution time, rows read/scanned, join count, query plan, index size, summary-table size and load/refresh time. Use EXPLAIN or the DBMS query-plan facility.

Query / rows	OLTP time	Star time	Star + summary time	Rows read / joins / notes
Q1 – 1K				
Q1 – 10K				
Q1 – 100K				
Q1 – 1M				
Q2 – 1K				
Q2 – 10K				
Q2 – 100K				
Q2 – 1M				
Q3 – 1K				
Q3 – 10K				
Q3 – 100K				
Q3 – 1M				

The table is intentionally left blank because execution times and rows scanned depend on the student's hardware, DBMS, indexes, data distribution and cache. Do not report invented timings as experimental evidence.

8.4 Indexes and summary structures

Useful indexes include FactSales(DateKey), FactSales(StoreKey), FactSales(ProductKey), FactSales(CustomerKey), and composite indexes selected from actual filter and grouping patterns. A summary table can store monthly net sales by StoreKey, ProductKey or Department and Region. Indexes reduce lookup and filtering cost; summaries reduce the number of fact rows that need aggregation.

Expected explanation: the OLTP schema often needs more joins and may scan transaction tables. The star schema reduces joins and is easier for analytical optimisers. The indexed/summary design can be fastest for

repeated queries, but requires additional storage, index maintenance and refresh work. A summary may be stale between refreshes and cannot answer every arbitrary detailed query.

8.5 What to include as evidence

- Screenshots or exported EXPLAIN plans for all three query versions.
- A graph of median query time against fact-row count.
- Rows scanned, join count and storage size, not only elapsed time.
- Summary-table creation and refresh time, plus a reconciliation proving its totals match FactSales.

TASK 9 – BUILD A KNIME REPORTING WORKFLOW (3 MARKS)

9.1 Proposed node sequence

```
CSV Reader / DB Connector
       |
Column Filter -> String Manipulation / Date&Time Parser
       |
Missing Value -> Duplicate Row Filter -> Rule-based Row Filter
       |
Joiner (dimension lookups, if source is not already warehouse-ready)
       |
GroupBy (SUM net sales, SUM quantity, COUNT orders)
       |
Pivoting (quarters or regions as columns)
       |
Bar Chart / Line Plot / Table View
       |
Excel Writer / PDF or report export
```

9.2 Node purpose and concept mapping

KNIME node	Purpose	Warehouse / OLAP concept
CSV Reader / DB Reader	Read fact and dimension data.	Extraction / source access.
Column Filter	Keep required fields only.	Projection and ETL preparation.
String Manipulation	Trim, standardise case and clean labels.	Data cleansing.
Date&Time Parser	Convert source date strings to date values.	Data type standardisation.
Missing Value	Replace or flag missing values according to rules.	Data-quality handling.
Duplicate Row Filter	Remove exact duplicate source rows where business key confirms duplication.	Deduplication.
Row Filter	Select dates, stores, regions or valid quantities.	OLAP slice/dice.
Joiner	Join facts with dimensions.	Dimensional key resolution / relational join.
GroupBy	Sum sales and quantity by selected dimensions.	Aggregation / roll-up.
Pivoting	Convert grouped values into report columns.	Pivot operation.
Chart nodes	Display trends and comparisons.	BI reporting.
Excel Writer	Export final management report.	Report delivery.

9.3 Mapping to ETL and OLAP; limitation

Reader, parser, string manipulation, missing-value and duplicate nodes correspond to extraction and transformation in Task 5. Row Filter represents slice/dice, GroupBy represents roll-up, and Pivoting represents pivot in Task 6. Joiner connects fact and dimension data. Charts and Excel Writer create the management report.

A limitation at 1,000,000+ rows is that KNIME workflows may require substantial memory and can become slower when large datasets are fully loaded into memory. Performance depends on node implementation and hardware. Use database-side filtering and aggregation, streaming-capable nodes, batching and DB connections rather than pulling all raw rows into memory.

Evidence required: provide a screenshot of the executed workflow, a sample output report and node annotations. The node sequence above is the workflow design to implement in KNIME.

TASK 10 – RECOMMEND AND DEFEND THE FINAL DESIGN (3 MARKS)

10.1 Final recommendation

Recommend a central enterprise data warehouse with dependent data marts, a star schema for FactSales and FactReturns, conformed Date/Product/Store/Customer dimensions, and a small set of materialised OLAP summaries. This balances analytical speed, business-user simplicity, historical reporting and governance.

10.2 Why this is better than alternatives

- Compared with direct OLTP reporting, the warehouse isolates analytical workload and avoids slowing billing counters.
- Compared with independent marts, the EDW provides consistent net-sales, customer and product definitions.
- Compared with a fully snowflaked design, the star schema requires fewer joins and is easier for analysts and BI tools.
- Compared with a virtual-only warehouse, persistent warehouse storage preserves history and supports predictable analytical performance.

10.3 Evidence and trade-offs

Task 4's illustrative storage estimate shows that snowflaking may reduce dimension storage, but the saving is modest for the given dimension sizes while increasing joins. Task 8 must provide measured query plans, row counts and median timings to prove the performance benefit of the star schema and selected summaries. The final submission should quote the student's actual measured results, not assume a particular speed-up.

Trade-offs include initial ETL and storage cost, delayed data compared with live OLTP, the need to maintain SCD history, and extra refresh cost for indexes and summary tables. At 1,000,000+ sales lines, a fact table with partitioning by date, suitable indexes, batch loading and selective aggregates remains practical on an appropriately sized relational warehouse. Further growth may require columnar storage, distributed processing or a cloud warehouse.

TASK 11 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

A. Impact analysis (2 marks)

The online channel adds about 20,000 orders per day, a different product-code format and a DeliveryPartner attribute. Customer address and loyalty tier change frequently, and executives require hourly dashboards that report sales using the address and tier at the time of the order.

Area	Affected?	Impact
Source integration	Yes – structural and process	Add online source connector, source

		identifier and source-specific product-code mapping.
DimProduct	Yes – structure/ETL	Retain a canonical ProductCode and a cross-reference from each source system's code to ProductKey.
DimCustomer	Yes – history	SCD Type 2 for address and loyalty tier; preserve effective dates and current flag.
FactSales	Yes – structure	Add ChannelKey and DeliveryPartnerKey (or a delivery partner dimension) and retain order source identifiers.
DimStore	Partly	Online orders may not have a physical store; add an Online/Unknown fulfilment location member or fulfilment-location dimension.
ETL	Yes – process/performance	Use CDC or micro-batch ingestion, deduplication and idempotent loads; complete hourly refresh.
OLAP cubes	Partly	Add Channel and Delivery Partner analysis; existing time/product/store/customer hierarchies remain useful.
Architecture	Yes – performance	Increase staging, compute and storage capacity; introduce hourly orchestration and monitoring.

The basic sales-line grain and core dimensions do not need to be discarded. The design should be modified rather than fully redesigned.

B. Evaluation of the original design (2 marks)

At 20,000 orders per day, the new channel creates 140,000 online orders per week and approximately 600,000 orders in a 30-day month. If an order contains an average of 3 line items, the online channel adds about 60,000 fact rows per day, or 1.8 million fact rows per 30-day month. This is an assumption for capacity planning; the real average line count must be measured.

A daily batch-only design is inadequate for an hourly dashboard because new online sales may remain invisible until the next daily load. A design that uses a single source product code may also fail to match the new channel's code format. Overwriting customer address or loyalty tier would fail the requirement to analyse sales by the historical values. The original star schema is still adequate for sales analysis, but ETL cadence, source mapping and dimension history must be extended.

C. Adaptation / revised design (2 marks)

Revised schema changes

```
DimProduct(ProductKey, CanonicalProductCode, ProductName, ...)
BridgeProductSource(SourceSystem, SourceProductCode, ProductKey)
DimCustomer(CustomerKey, CustomerID, Address, LoyaltyTier,
            EffectiveFrom, EffectiveTo, IsCurrent, ...)
DimChannel(ChannelKey, ChannelName)  -- Store / Online
DimDeliveryPartner(DeliveryPartnerKey, PartnerCode, PartnerName)
FactSales(..., CustomerKey, ProductKey, StoreKey, DateKey,
          PromotionKey, ChannelKey, DeliveryPartnerKey, measures...)
```

For online transactions, StoreKey can point to a dedicated Online Fulfilment/Not Applicable member if no physical store applies. DeliveryPartnerKey points to the delivery partner dimension. SourceSystem and SourceOrderID should be retained to support traceability and duplicate prevention.

Revised hourly ETL flow

```
Online source + store OLTP
        |
CDC / incremental extract every 5–15 minutes
        |
Raw staging with source, batch ID and event timestamp
        |
Validate -> map product codes -> resolve customer
        |
Apply SCD Type 2 changes (address / loyalty tier)
        |
Load dimensions -> resolve surrogate keys -> load facts
        |
Reconcile batch -> refresh hourly aggregates/cubes
        |
Publish dashboards; show last successful refresh time
```

A 5–15 minute micro-batch allows the warehouse to publish a complete hourly dashboard while providing time for validation and retry. The final dashboard refresh can run at the top of each hour. Late-arriving updates are handled through batch watermarks and controlled restatement.

D. Evidence-based defence and monitoring (2 marks)

The volume calculation indicates that the online channel may add approximately 60,000 fact rows per day under the three-lines-per-order assumption. The warehouse must therefore be tested with the existing store volume plus this incremental volume. Run hourly loads under peak conditions and measure extraction duration, transformation duration, fact insert rate, rejected rows, query latency, aggregate refresh duration and dashboard data freshness. Compare these values with the one-hour refresh target.

Metric to monitor	Reason / target approach
Orders and fact rows per batch	Detect volume spikes and confirm source completeness.
End-to-end load duration	Must finish within the hourly refresh window with recovery time.
Dashboard data age	Should be no more than the agreed one-hour service target.
Rejected / duplicate records	Detect mapping and source-quality failures.
Unknown ProductKey / CustomerKey count	Identify late or unmapped dimension members.
SCD Type 2 row growth	Detect excessive or incorrect customer history changes.
Query latency and rows scanned	Confirm dashboards remain responsive as facts grow.
Aggregate refresh time and storage	Ensure pre-computation remains cost-effective.

Final defence: modify and retain the central EDW and star schema. Add source-code mapping, Channel and Delivery Partner dimensions, SCD Type 2 customer history, incremental micro-batch ETL and hourly aggregate refresh. This preserves the original analytical model while addressing the new data source, historical reporting and freshness requirements. The decision should be confirmed by measured load and query tests before production deployment.

OVERALL CONCLUSION

The proposed FreshCart solution separates operational transactions from analytical reporting through a central enterprise data warehouse. A star schema provides a simple and efficient model for sales and returns analysis, while ETL standardises source data and preserves customer history. OLAP cuboids and operations support management questions at multiple levels, and KNIME or a Python-based application can present the results. Selective indexes and aggregate tables improve frequently used queries but introduce storage and refresh costs. When the online channel is introduced, the warehouse should be extended with source-code mapping, delivery

partner and channel dimensions, SCD Type 2 history and hourly micro-batch processing. The final implementation must validate performance using actual measurements and reconciliation results.

REFERENCES

- 1. CSA16 Data Warehousing and Data Mining, Assignment 1 – CO1, FreshCart Retail Analytics scenario (user-provided assignment brief).
- 2. Ralph Kimball and Margy Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling.
- 3. W. H. Inmon, Building the Data Warehouse.
- 4. KNIME Analytics Platform documentation – nodes for CSV Reader, Joiner, GroupBy, Pivoting and reporting.
- 5. PostgreSQL / selected DBMS documentation for SQL DDL, indexes and EXPLAIN query plans.